BINARY SEARCH

The Guessing Game | O(log n)

🎯 How to Recognize Binary Search

Core Philosophy:

Binary Search is NOT just for sorted arrays. It's a **"Guessing Game"** based on **MONOTONICITY**.

The 4 Criteria

1 Single Answer Find ONE specific value (index, minimum speed, maximum distance)	2 Search Space Answer lies within a known range (Low to High)	3 Optimization Find MINIMUM or MAXIMUM value that satisfies condition	4 Monotonic Function Results follow pattern: NO NO NO YES YES YES (or vice versa)

🔥 The Turning Point

Guess:	1	2	3	4	5	6	7
Result:	NO	NO	NO	YES	YES	YES	YES

↑ We're looking for the TURNING POINT (First YES or Last NO)

💡 The Fan Regulator Example

Scenario: Find minimum fan speed (1-7) where you feel comfortable (not hot) to save electricity.

- Speeds 1, 2, 3: **HOT (NO)**
- Speeds 4, 5, 6, 7: **COMFORTABLE (YES)**
- **Goal:** Find FIRST YES (Speed 4) = Minimum that works

📋 The Universal Template

Two Patterns:

Pattern A: Find MINIMUM (First YES)

→ Koko Bananas, Book Allocation, Minimum Speed

→ When condition is TRUE, try SMALLER (high = mid - 1)

Pattern B: Find MAXIMUM (Last YES)

→ Aggressive Cows, Maximum Distance

→ When condition is TRUE, try LARGER (low = mid + 1)

Template Code

```
// PATTERN A: Find MINIMUM (First YES)
int low = MIN_POSSIBLE, high = MAX_POSSIBLE;
int ans = high; // Default to worst case

while (low <= high) {
    int mid = low + (high - low) / 2;

    if (check(mid)) { // If condition is satisfied (YES)
        ans = mid;        // Store answer
        high = mid - 1;  // Try SMALLER (minimize)
    } else {
        low = mid + 1;   // Condition not met, go LARGER
    }
}
return ans;

// PATTERN B: Find MAXIMUM (Last YES)
int low = MIN_POSSIBLE, high = MAX_POSSIBLE;
int ans = low; // Default to worst case

while (low <= high) {
    int mid = low + (high - low) / 2;

    if (check(mid)) { // If condition is satisfied (YES)
        ans = mid;       // Store answer
        low = mid + 1; // Try LARGER (maximize)
    } else {
        high = mid - 1; // Condition not met, go SMALLER
    }
}
return ans;
```

Problem 1: Aggressive Cows

Problem: Place K cows in stalls such that the MINIMUM distance between any two cows is MAXIMIZED.

Pattern: Find MAXIMUM (Last YES)

🎯 The Logic

We **GUESS** a distance:
- If we can place cows with distance X → Try LARGER distance
- If we cannot → Distance too large, try SMALLER

📊 Monotonicity Visualization

Stalls: [1, 2, 4, 8, 9] | K = 3 cows

```
Distance Guess:  1    2    3    4    5    6
Can Place?:     YES  YES  YES  NO   NO   NO
                           ↑
                 Last YES = Answer (3)

With distance 3:
Cow 1 at position 1
Cow 2 at position 4 (1+3)
Cow 3 at position 8 (4+4 ≥ 4+3)
✓ Can place all 3 cows!
```

💻 Java Code

```java
/**
 * Aggressive Cows
 * Pattern: Find MAXIMUM (Last YES)
 *
 * Time: O(n log n + n log(max-min))
 * Space: O(1)
 */
import java.util.Arrays;

public class AggressiveCows {
    // Helper: Check if we can place K cows with at least 'dist' gap
    static boolean canPlace(int[] stalls, int k, int dist) {
        int cows = 1; // Place first cow at first stall
        int lastPos = stalls[0];

        for (int i = 1; i < stalls.length; i++) {
            if (stalls[i] - lastPos >= dist) {
                cows++;
                lastPos = stalls[i];
            }
            if (cows >= k) return true;
        }
        return false;
    }

    public static int solve(int[] stalls, int k) {
        Arrays.sort(stalls); // Must sort first!

        int low = 1;
        int high = stalls[stalls.length - 1] - stalls[0];
        int ans = 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (canPlace(stalls, k, mid)) {
                ans = mid;       // Store answer
                low = mid + 1;  // Try LARGER distance (maximize)
            } else {
                high = mid - 1; // Distance too large
            }
        }

        return ans;
    }
}
```

Problem 2: Koko Eating Bananas

Problem: Find MINIMUM eating speed K to finish all banana piles within H hours.

LeetCode: #875

Pattern: Find MINIMUM (First YES)

🎯 The Logic

Range: 1 to Max(pile size)
Monotonicity: If speed S works, any speed > S also works
Goal: Find FIRST speed that works (minimum)

📊 Monotonicity Visualization

Piles: [3, 6, 7, 11] | H = 8 hours

```
Speed Guess:    1    2    3    4    5    6   ...
Finish in 8h?: NO   NO   NO  YES  YES  YES ...
                              ↑
                   First YES = Answer (4)

Calculation for speed 4:
Pile 3  → ceil(3/4)  = 1 hour
Pile 6  → ceil(6/4)  = 2 hours
Pile 7  → ceil(7/4)  = 2 hours
Pile 11 → ceil(11/4) = 3 hours
                     -------
Total: 8 hours ≤ 8 ✓
```

💻 Java Code

```java
/**
 * Koko Eating Bananas
 * LeetCode 875
 * Pattern: Find MINIMUM (First YES)
 *
 * Time: O(n log(max))
 * Space: O(1)
 */
public int minEatingSpeed(int[] piles, int h) {
    // Find max pile size
    int max = 0;
    for (int p : piles) {
        max = Math.max(max, p);
    }

    int low = 1;
    int high = max;
    int ans = max; // Worst case: eat fastest pile in 1 hour each

    while (low <= high) {
        int mid = low + (high - low) / 2;

        // Calculate hours needed at speed 'mid'
        long hours = 0;
        for (int pile : piles) {
            hours += (pile + mid - 1) / mid; // Ceil division
        }

        if (hours <= h) {
            ans = mid;        // Works, try SMALLER speed (minimize)
            high = mid - 1;
        } else {
            low = mid + 1;  // Too slow, go faster
        }
    }

    return ans;
}
```

Problem 3: Book Allocation Problem

Problem: Allocate books to K students such that the MAXIMUM pages assigned to any student is MINIMIZED. Books must be allocated contiguously.

Pattern: Find MINIMUM (First YES)

🎯 The Logic

We **GUESS** a page limit:
- If we can allocate books with this limit → Try SMALLER limit
- If we need more than K students → Limit too small, increase it

Range: Max(single book) to Sum(all books)

📊 Monotonicity Visualization

Books: [12, 34, 67, 90] | K = 2 students

```
Limit Guess:   ... 100   110   113   120   130 ...
Can Allocate?: ... NO    NO    YES   YES   YES ...
                                ↑
                    First YES = Answer (113)

With limit 113:
Student 1: [12, 34, 67] = 113 pages ✓
Student 2: [90] = 90 pages ✓
Total: 2 students = K ✓
```

💻 Java Code

```java
/**
 * Book Allocation Problem
 * Pattern: Find MINIMUM (First YES)
 *
 * Time: O(n log(sum))
 * Space: O(1)
 */
public class BookAllocation {
    // Helper: Can we allocate to K students with max 'maxPages' limit?
    static boolean isPossible(int[] books, int students, int maxPages) {
        int count = 1; // Start with first student
        int currentSum = 0;

        for (int pages : books) {
            if (pages > maxPages) return false; // Single book too large

            if (currentSum + pages <= maxPages) {
                currentSum += pages;
            } else {
                count++; // Assign to next student
                currentSum = pages;
                if (count > students) return false;
            }
        }
        return true;
    }

    public static int findPages(int[] books, int students) {
        if (students > books.length) return -1;

        int low = 0;
        int high = 0;

        for (int p : books) {
            low = Math.max(low, p); // At least the biggest book
            high += p;              // At most all books to one student
        }

        int ans = high;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (isPossible(books, students, mid)) {
                ans = mid;        // Possible, try SMALLER limit
                high = mid - 1;
            } else {
                low = mid + 1;  // Limit too small
            }
        }

        return ans;
    }
}
```

Problem 4: Peak Index in Mountain Array

Problem: Find the peak element in an array that increases then decreases (mountain shape).

LeetCode: #852

Pattern: Standard Binary Search (comparing slopes)

🎯 The Logic (Slope Analysis)

Compare arr[mid] with arr[mid+1]:
- If arr[mid] < arr[mid+1] → **Rising slope (/) → Peak is RIGHT**
- If arr[mid] > arr[mid+1] → **Falling slope (\) → Peak is LEFT or CURRENT**

📊 Visual Diagram

Array: [1, 3, 8, 12, 10, 6, 2]

```
        Peak (12)
          /\
         /  \
        /    \
       /      \
      /        \
     /          \
    1  3  8  12  10  6  2
    0  1  2  3   4   5  6

Mid = 3 (value 12):
arr[3] = 12 > arr[4] = 10
Falling slope! Peak is LEFT or CURRENT
high = mid
```

💻 Java Code

```java
/**
 * Peak Index in Mountain Array
 * LeetCode 852
 *
 * Time: O(log n)
 * Space: O(1)
 */
public int peakIndexInMountainArray(int[] arr) {
    int low = 0;
    int high = arr.length - 1;

    while (low < high) { // Note: low < high (not <=)
        int mid = low + (high - low) / 2;

        if (arr[mid] < arr[mid + 1]) {
            // Rising slope: /
            // Peak is to the RIGHT
            low = mid + 1;
        } else {
            // Falling slope: \
            // Peak is LEFT or CURRENT
            high = mid; // Keep mid as it could be peak
        }
    }

    return low; // Peak found
}
```

Problem 5: Search in Rotated Sorted Array

Problem: Search for target in a sorted array that has been rotated.

LeetCode: #33

Example: [4, 5, 6, 7, 0, 1, 2], target = 0 → index 4

🎯 The Logic (Two Parts)

Array has **Part 1 (large values)** and **Part 2 (small values)**.
One half is ALWAYS sorted. Identify which half, then check if target is in that range.

📊 Visual Diagram

Array: [4, 5, 6, 7, 0, 1, 2]

```
    Part 1 (Large)    Part 2 (Small)
      /7
     /6
    /5
   /4
               /2
              /1
             /0

Mid = 3 (value 7):
arr[low]=4 <= arr[mid]=7 → Left half SORTED
Is target 0 in [4, 7]? NO
So search RIGHT half
```

💻 Java Code

```java
/**
 * Search in Rotated Sorted Array
 * LeetCode 33
 *
 * Time: O(log n)
 * Space: O(1)
 */
public int search(int[] nums, int target) {
    int low = 0;
    int high = nums.length - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (nums[mid] == target) return mid;

        // Identify which part is sorted
        if (nums[low] <= nums[mid]) {
            // LEFT half is sorted
            if (nums[low] <= target && target < nums[mid]) {
                // Target is in the sorted LEFT range
                high = mid - 1;
            } else {
                // Target is in RIGHT (unsorted) part
                low = mid + 1;
            }
        } else {
            // RIGHT half is sorted
            if (nums[mid] < target && target <= nums[high]) {
                // Target is in the sorted RIGHT range
                low = mid + 1;
            } else {
                // Target is in LEFT (unsorted) part
                high = mid - 1;
            }
        }
    }

    return -1; // Not found
}
```

Problem 6: Find Minimum in Rotated Array

Problem: Find the minimum element (pivot point) in a rotated sorted array.

LeetCode: #153

Example: [4, 5, 6, 7, 0, 1, 2] → 0

🎯 The Logic

Compare arr[mid] with arr[high]:
- If arr[mid] > arr[high] → Mid is in Part 1 (large), min is RIGHT
- If arr[mid] <= arr[high] → Mid is in Part 2 (small), min is LEFT or CURRENT

📊 Visual Diagram

Array: [4, 5, 6, 7, 0, 1, 2]

```
Compare mid with high:

Mid = 3 (value 7), High = 6 (value 2)
arr[mid]=7 > arr[high]=2
→ Mid is in Part 1 → Min is RIGHT
low = mid + 1

Mid = 5 (value 1), High = 6 (value 2)
arr[mid]=1 <= arr[high]=2
→ Mid is in Part 2 → Min is LEFT or CURRENT
high = mid
```

💻 Java Code

```java
/**
 * Find Minimum in Rotated Sorted Array
 * LeetCode 153
 *
 * Time: O(log n)
 * Space: O(1)
 */
public int findMin(int[] nums) {
    int low = 0;
    int high = nums.length - 1;

    while (low < high) { // Note: low < high (not <=)
        int mid = low + (high - low) / 2;

        // Compare mid with high to detect which part we're in
        if (nums[mid] > nums[high]) {
            // We are in Part 1 (Big values)
            // Minimum is to the RIGHT
            low = mid + 1;
        } else {
            // We are in Part 2 (Small values)
            // Minimum is LEFT or CURRENT
            high = mid;
        }
    }

    return nums[low];
}
```

Problem 7: Search in 2D Matrix (Fully Sorted)

Problem: Search in a matrix where each row is sorted AND the last element of row i < first element of row i+1.

LeetCode: #74

Pattern: Treat as 1D array (flatten conceptually)

🎯 The Logic (Coordinate Mapping)

Matrix is effectively one long sorted array!
Mapping: For 1D index mid:
- row = mid / cols
- col = mid % cols

📊 Visual Diagram

Matrix 4×4 (indices 0 to 15)

```
Matrix:       1D Index:
 1   3   5   7      0   1   2   3
10  11  16  20      4   5   6   7
23  30  34  60      8   9  10  11
61  62  63  65     12  13  14  15

Mid = 9:
row = 9 / 4 = 2
col = 9 % 4 = 1
Check matrix[2][1] = 30
```

💻 Java Code

```java
/**
 * Search 2D Matrix (Fully Sorted)
 * LeetCode 74
 *
 * Time: O(log(n*m))
 * Space: O(1)
 */
public boolean searchMatrix(int[][] matrix, int target) {
    int n = matrix.length;        // Rows
    int m = matrix[0].length;    // Cols

    int low = 0;
    int high = (n * m) - 1; // Imaginary 1D array

    while (low <= high) {
        int mid = low + (high - low) / 2;

        // Map 1D index back to 2D coordinates
        int row = mid / m;
        int col = mid % m;

        if (matrix[row][col] == target) {
            return true;
        }

        if (matrix[row][col] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return false;
}
```

Problem 8: Search 2D Matrix II (Row/Col Sorted)

Problem: Search in a matrix where rows and columns are sorted independently (but NOT row-to-row sorted).

LeetCode: #240

Pattern: "Staircase" Search (Start from corner)

🎯 The Logic (Elimination)

Start at **TOP-RIGHT corner** (or bottom-left):
- If current > target → Column too big, move LEFT
- If current < target → Row too small, move DOWN
- If current == target → Found!

📊 Visual Diagram

Find target = 5

```
Matrix:
 1   4   7  11 ← Start here
 2   5   8  12
 3   6   9  16

Step 1: curr=11 > 5 → Move LEFT
 1   4   7  11
 2   5   8  12
 3   6   9  16

Step 2: curr=7 > 5 → Move LEFT
 1   4   7  11
 2   5   8  12
 3   6   9  16

Step 3: curr=4 < 5 → Move DOWN
 1   4   7  11
 2   5   8  12 ← FOUND!
 3   6   9  16
```

💻 Java Code

```java
/**
 * Search 2D Matrix II (Staircase Search)
 * LeetCode 240
 *
 * Time: O(n + m)
 * Space: O(1)
 */
public boolean searchMatrix(int[][] matrix, int target) {
    if (matrix == null || matrix.length == 0) return false;

    int n = matrix.length;
    int m = matrix[0].length;

    // Start at TOP-RIGHT corner
    int row = 0;
    int col = m - 1;

    while (row < n && col >= 0) {
        if (matrix[row][col] == target) {
            return true;
        }

        if (matrix[row][col] > target) {
            col--; // Eliminate column (too big)
        } else {
            row++; // Eliminate row (too small)
        }
    }

    return false;
}
```

Problem 9: K-th Smallest Element in Sorted Matrix

Problem: Find the K-th smallest number in a row/column sorted matrix.

LeetCode: #378

Pattern: Binary Search on ANSWER RANGE (not indices!)

🎯 The Logic (Binary Search on Values)

Instead of searching indices, search the **VALUE RANGE**:
- Range: matrix[0][0] to matrix[n-1][n-1]
- For each guess, **COUNT** how many elements ≤ guess
- If count < k → Answer is larger
- If count ≥ k → This might be answer, try smaller

📊 Visual Diagram

Matrix (sorted rows/cols), K=8

```
Matrix:
 1   5   9
10  11  13
12  13  15

Range: [1 ... 15]

Guess mid = 11:
Count elements ≤ 11 using staircase:
→ Start top-right, count = 6

count=6 < k=8 → Answer must be LARGER
low = mid + 1

Guess mid = 13:
Count ≤ 13 = 8
count=8 >= k=8 → This could be answer
high = mid - 1 (try smaller)
```

💻 Java Code

```java
/**
 * K-th Smallest Element in Sorted Matrix
 * LeetCode 378
 *
 * Time: O(n log(max-min))
 * Space: O(1)
 */
public class KthSmallestMatrix {
    // Helper: Count elements <= target using staircase method
    static int countLessEqual(int[][] matrix, int target) {
        int n = matrix.length;
        int count = 0;

        // Start bottom-left corner
        int row = n - 1;
        int col = 0;

        while (row >= 0 && col < n) {
            if (matrix[row][col] <= target) {
                count += (row + 1); // Add entire column segment
                col++;
            } else {
                row--;
            }
        }

        return count;
    }

    public int kthSmallest(int[][] matrix, int k) {
        int n = matrix.length;

        int low = matrix[0][0];           // Min element
        int high = matrix[n-1][n-1];     // Max element
        int ans = low;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            int count = countLessEqual(matrix, mid);

            if (count >= k) {
                ans = mid;        // Possible answer
                high = mid - 1;  // Try smaller
            } else {
                low = mid + 1;   // Too small
            }
        }

        return ans;
    }
}
```

📝 Quick Reference

Problem Types

```
┌───────────────────────────┬────────────┬─────────────────┐
│ Problem                   │ Pattern    │ Range           │
├───────────────────────────┼────────────┼─────────────────┤
│ Aggressive Cows           │ Max (Last Y)│ 1 to max-min   │
│ Koko Bananas              │ Min (First Y)│ 1 to max(pile)│
│ Book Allocation           │ Min (First Y)│ max(arr) to sum│
│ Search Rotated Array      │ Standard   │ 0 to n-1        │
│ Peak Mountain Array       │ Standard   │ 0 to n-1        │
│ Search 2D Matrix (Sorted) │ Standard   │ 0 to n*m-1      │
│ K-th Smallest (Matrix)    │ Min (First Y)│ min to max value│
└───────────────────────────┴────────────┴─────────────────┘
```

Key Takeaways

- **Monotonicity** is the key - look for NO NO NO YES YES pattern
- **Two patterns:** Find MINIMUM (move high on YES) or MAXIMUM (move low on YES)
- **Helper function:** Write check(mid) to test if guess is valid
- **Time complexity:** O(log n) searches × O(n) check = O(n log n)
- **Always store answer:** Don't assume low/high will be answer at end